Task2View Pilot Study
Subjects, Models, and Experimental Conditions

September 2026

1 Pilot study

This pilot checks whether a stakeholder goal and a repository can be turned into one architecture view, before a larger evaluation. Three conditions receive the same two systems and the same ten goals. The diagrams are in `comparison/`: folder 1 is the agent with the skill, folder 2 is the plain agent, and folder 3 is Task2View. Folder 3 is the image set from Task2View architecture. Figure 1 is the whole pilot: repositories, stakeholder queries, the three approaches with the model each one used, the views, and the review. Section 2 is that last step. The Task2View diagrams were not in the review.

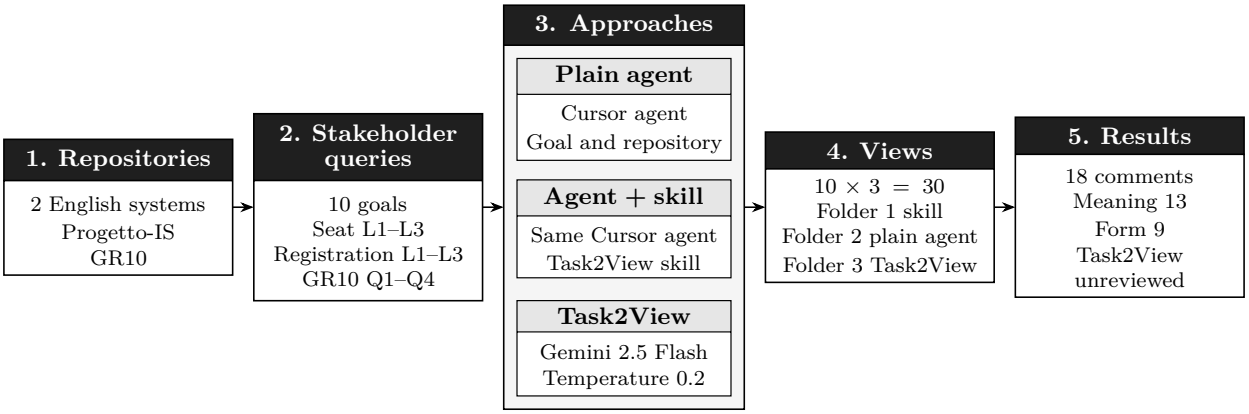

Figure 1: Pilot study flow.

1.1 Subjects

Both subjects are the English variants under `data_points/`. Each fits in one model call. The ten goals are a developer changing seat booking and a tester covering registration, each at three levels of detail, plus four GR10 roles: architect, DevOps engineer, database engineer, and business analyst.

1.2 Models

Task2View uses one model for every phase call. The two agent conditions use one Cursor general-purpose agent. The skill is the only difference between those two arms. The agent run folders do not store a model id.

	Progetto-IS	GR10
System	University library seat reservation	Municipal report management
Stack	Java 21, Swing + Spring Boot, JPA, MySQL	Java 23, Swing, JPA, MySQL; BCED packages
Size	80 Java files, ∼10k LOC	44 Java files, ∼11.8k LOC
Goals	Seat booking L1–L3; registration L1–L3	Q1–Q4
Reference	Design documentation	Visual Paradigm model

Table 1: Pilot subjects.

Condition	Model	Setting	Role
Task2View	Gemini 2.5 Flash	temperature 0.2	Every phase that calls a model
Agent, both arms	Cursor general-purpose agent	same model in both arms	Reads the repository and writes the view

Table 2: Models in the pilot. Gemini is the Google AI Studio default (`GEMINI_MODEL`, otherwise `gemini-2.5-flash`).

1.3 Conditions

The repository and the goal stay fixed.

Plain agent (`comparison/folder2`). The agent receives the repository path and the goal. It is given neither the skill, the viewpoint catalog, nor Task2View. It writes a PlantUML view, and on a few goals a short note.

Agent with the Task2View skill (`comparison/folder1`). The same kind of agent follows `task2view-skill`. It interprets the stakeholder, chooses one catalog viewpoint, binds that viewpoint's grain, and publishes the phase JSON plus a diagram. It does not run `task2view`.

Task2View (`comparison/folder3`, `runs/pilot`). PageRank on the uses/calls graph (Aider RepoMap, damping 0.85) proposes files, and a token budget keeps a subset. This batch kept 2–8 files. The phases are separate Gemini calls:

1. interpret the stakeholder and task;
2. write architectural questions;
3. choose one catalog viewpoint and its diagram language;
4. build the graph, scope files, and write an analysis plan;
5. extract claims from the files named in that plan;
6. reconcile questions with the claims, without redrawing the view;
7. publish a view model from that evidence;
8. validate names against the scoped graph and emit PlantUML.

Each pilot run records `scope_strategy: pagerank` and an `architecture_evidence_model.json`.

	Plain agent	Agent + skill	Task2View
Who decides	The agent	One agent, one catalog viewpoint	A Gemini call per phase
Repository	Read as the agent chooses	Read after the viewpoint is fixed	PageRank, then a token budget (2–8 files here)
Output	Diagram, sometimes a note	Phase JSON + diagram	Phase JSON, evidence model, diagram, validation report
Model	Cursor agent	Same Cursor agent	Gemini 2.5 Flash, temperature 0.2

Table 3: The three pilot conditions.

2 Results

A reviewer wrote one comment on each of the ten goals for the skill agent and the plain agent, plus one comment covering all three seat-booking diagrams of the plain agent: 18 comments in `comparison_review.txt`. Seven fit the goal, five are partial, and six miss it. Short goals are the ones accepted. The detailed stakeholder queries are the ones rejected. The Task2View diagrams in folder 3 were not reviewed.

The seven themes fall into two buckets. **Meaning** is whether the diagram says the right architectural thing. **Form** is whether it is drawn as a proper view. Thirteen comments raise a meaning issue. Nine raise a form issue. Six raise both. Two plain-agent diagrams raise neither: the DevOps deployment view, and registration at the shortest stakeholder query.

Bucket	Theme	Skill	Plain agent
Meaning	Second view: structure and behavior in one picture	3	3
Meaning	Grain: too coarse, or several levels at once	2	3
Meaning	Layers: components with no package around them	4	0
Meaning	Names: not a source element, or parts the goal did not ask for	1	3
Form	Notation: boxes, symbols, stereotypes	2	4
Form	Placement: a fact drawn on the wrong mark	2	0
Form	Layout: arrows or arrangement, content otherwise accepted	1	1

Table 4: Comment blocks that raise each theme. One comment can raise more than one theme. Counts are out of 10 skill comments and 8 plain-agent comments.

Meaning. *Second view* (6). The reviewer wants a component diagram for the parts and a sequence diagram for the order of calls. Registration L2–L3 and seat booking L2–L3, in both conditions. *Grain* (5). The level of detail does not follow the stakeholder query. The longest registration request to the skill agent produced the coarsest picture. The plain agent mixes code snippets with subsystems. *Layers* (4). The boxes are a flat list. The reviewer wants Boundary, Control, Entity, and Database around them. All four comments are on the skill agent. The architect comment asks for the GUI pieces to sit inside one higher package. *Names* (4). A box is not a source element, or the diagram adds a part the goal did not ask for. Three of the four are the plain agent. Registration L2 draws a box named "boundary Autenticazione POST /api/auth/registrazione".

Form. *Notation* (6). The content may be useful, and it is not drawn as the view it claims to be. Four of the six are the plain agent. The architect diagram is closer to a flowchart than a component view. The skill agent's database diagram uses aggregation symbols for the wrong relation. *Placement* (2). The fact is in the diagram and attached to the wrong mark. Both comments are on the skill agent. Responsibilities are written on edges. On the database goal, `id_cittadino` is the label of a line between two classes, where the reviewer expects an attribute. *Layout* (2). The content is accepted. An arrow has no source on the skill agent's deployment view, and the plain agent's analyst diagram is hard to follow.

The skill agent is the condition that drops layers and misplaces facts. The plain agent is the condition that drops notation and grounded names. On registration at the shortest stakeholder query, and on the persistence core of the database goal, the plain agent is the diagram the reviewer prefers.

Three skill comments on Progetto-IS name the other feature. Registration L1 and L2 are discussed as seat booking. Seat booking L3 is discussed as registration. The theme in each comment still holds. Those three sentences should be checked against the diagram before they are cited for a specific goal.